Gradient Boosting Regression (GBR)

Complete Interview & Revision Notes

Introduction

Gradient Boosting Regression (GBR) is an ensemble learning technique that combines multiple weak learners (typically shallow decision trees) *sequentially* to create a strong predictive model.

Key ideas:

- **Sequential Corrections:** Each new tree learns to correct the mistakes (residuals) made by the current ensemble model.
- **Dependency:** Unlike Random Forest, where trees are built independently and in parallel, Gradient Boosting builds trees sequentially — each tree depends on all preceding ones.

Core Intuition

Suppose we want to predict house prices.

1. After making an initial prediction, there will be some errors.
2. Instead of building another tree from scratch to predict the price, Gradient Boosting trains the next tree specifically to **learn these errors (residuals)**.
3. Each new tree focuses only on what the previous ensemble failed to learn.

Mathematical Formulation

Let the training dataset be:

$$D = \{(x_1, y_1),\ (x_2, y_2),\ \ldots,\ (x_n, y_n)\}$$

where x_i are the input features and y_i are the actual target values.

Our objective is to learn an ensemble function $F(x)$ such that $F(x) \approx y$ by minimising a specified loss function $L(y,\ F(x))$.

Step 1 — Initial Prediction

For regression with Mean Squared Error (MSE), the optimal initial prediction that minimises the loss is the **mean** of the target values:

$$F_0(x) = \frac{1}{N} \sum_{i=1}^{N} y_i$$

Every sample initially receives this same baseline prediction.

Step 2 — Compute Residuals

Calculate the error made by the current model for each sample i:

$$r_{i1} = y_i - F_0(x_i)$$

Residuals represent the magnitude and direction by which the current predictions are incorrect.

Step 3 — Train the First Tree

Train a decision tree where:

- **Input:** X
- **Target:** r_{i1} (the residuals)

The tree learns a mapping $h_1(x)$ that approximates these residuals.

Step 4 — Update Predictions

Add a scaled fraction of the tree's prediction to the current model using a learning rate η:

$$F_1(x) = F_0(x) + \eta \cdot h_1(x), \qquad 0.01 \leq \eta \leq 0.3 \text{ (typical)}$$

Step 5 — Compute New Residuals

Using the updated model $F_1(x)$, compute the new pseudo-residuals:

$$r_{i2} = y_i - F_1(x_i)$$

These residuals are generally smaller in magnitude than those of the first iteration.

Step 6 — Train the Second Tree and Repeat

Train another tree $h_2(x)$ to predict r_{i2}, then update:

$$F_2(x) = F_1(x) + \eta \cdot h_2(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x)$$

General Update Rule

After m trees, the prediction is:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Expanded form after M total trees:

$$\boxed{F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x)}$$

Why Is It Called "Gradient" Boosting?

A common misconception is that Gradient Boosting *only* learns residuals $(y - \hat{y})$. **This is true only when the MSE loss is used.** In the general case, Gradient Boosting performs **gradient descent in functional space**: each new tree learns the **negative gradient of the loss function** with respect to the current predictions.

General Pseudo-Residual Formula

At boosting step m, the pseudo-residuals are:

$$r_{im} = -\left[\frac{\partial L(y_i,\, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$$

Proof for MSE Loss

Let the loss function be:

$$L(y,\, F(x)) = \frac{1}{2}\Big(y - F(x)\Big)^2$$

Differentiating with respect to $F(x)$:

$$\frac{\partial L}{\partial F(x)} = -\Big(y - F(x)\Big)$$

Taking the negative gradient:

$$-\frac{\partial L}{\partial F(x)} = y - F(x) = \text{Residual}$$

Key Interview Takeaway: Residuals are a special case of negative gradients — they are equivalent *only* when the loss function is MSE. For MAE loss, the negative gradient becomes $\text{sign}(y - F(x))$, meaning the tree fits the *direction* of the error ($+1$ or -1), not the raw difference.

Training Algorithm (Formal Steps)

1. **Initialise** the model with a constant value:

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i,\, \gamma)$$

2. **For** $m = 1$ **to** M (for each tree):

   (a) Compute pseudo-residuals (negative gradients):

$$r_{im} = -\left[\frac{\partial L(y_i,\, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)} \qquad \text{for } i = 1, \ldots, N$$

   (b) Fit a regression tree to the targets r_{im}, creating terminal regions (leaves) R_{jm} for $j = 1, \ldots, J_m$.

   (c) For each leaf R_{jm}, compute the optimal leaf value γ_{jm} that minimises the loss within that leaf:

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i,\, F_{m-1}(x_i) + \gamma)$$

   (d) Update the model:

$$F_m(x) = F_{m-1}(x) + \eta \sum_{j=1}^{J_m} \gamma_{jm}\, \mathbf{1}(x \in R_{jm})$$

Important Hyperparameters

Hyperparameter	Purpose
`n_estimators`	Total number of sequential trees to build.
`learning_rate`	(η) Scaling factor for each tree's contribution. Lower values require more trees but improve generalisation.
`max_depth`	Limits the depth of individual trees (controls weak-learner capacity; typically 3–8).
`min_samples_split`	Minimum number of samples required to split an internal node.
`subsample`	Fraction of training samples used to fit each base learner. Values < 1.0 implement *Stochastic* Gradient Boosting.

Overfitting Control

Because each tree aggressively targets the remaining errors, Gradient Boosting is susceptible to overfitting. The main mitigation strategies are:

- **Shrinkage (lower η):** Reduces the contribution of each individual tree, forcing the model to build more robust patterns over many iterations.
- **Early Stopping:** Monitors validation error and halts training when the validation score stops improving.
- **Subsampling (`subsample` < 1.0):** Trains each tree on a random subset of the data, introducing variance-reducing randomness.

Comparative Tables

Gradient Boosting vs. Random Forest

Aspect	Gradient Boosting	Random Forest
Training Structure	Sequential	Parallel / Independent
Primary Objective	Reduce bias sequentially	Reduce variance via averaging
Tree Complexity	Shallow (low variance, high bias)	Deep (high variance, low bias)
Overfitting Risk	High (requires careful tuning)	Low (robust to adding more trees)

Gradient Boosting vs. AdaBoost

Aspect	AdaBoost	Gradient Boosting
Correction Mechanism	Increases weights of misclassified samples	Fits negative gradient of the loss
Loss Flexibility	Fixed (exponential loss)	Flexible (MSE, MAE, Huber, cus
Optimisation Method	Heuristic weight updates	Numerical gradient descent

Vanilla Gradient Boosting vs. XGBoost

Aspect	Gradient Boosting (scikit-learn)	XGBoost
Regularisation	No native L_1/L_2 regularisation	Built-in L_1 (Lasso) & L_2 (Ridge)
Optimisation	First-order gradient expansion	Second-order Taylor expansion (gradients &
Hardware Usage	Strictly sequential execution	Feature-level parallelised tree construction

Key Interview Questions

Q1. Why is Gradient Boosting called "Gradient" Boosting?

A: Because it minimises the loss function by adding new base models sequentially using **gradient descent**. Each weak learner is trained to predict the negative gradient (pseudo-residuals) of the loss function evaluated at the current ensemble's predictions.

Q2. What happens if you set the learning rate to 1.0?

A: The model learns very quickly but will almost certainly overfit. It tracks training-data noise aggressively, causing variance to spike and degrading validation performance.

Q3. Why are shallow trees preferred over deep trees as weak learners?

A: Deep trees have low bias but high variance — they overfit quickly. Gradient Boosting's strength lies in reducing bias *sequentially*; starting with strong learners (deep trees) causes the boosting process to overfit almost immediately.

Q4. What is the difference between residuals and negative gradients?

A: Residuals $(y - \hat{y})$ are a specific instantiation of negative gradients and are equivalent only when the loss function is MSE. For other losses such as Huber or MAE, the negative gradients are computed differently and do not equal the raw residuals.

One-Line Definition

Gradient Boosting is a sequential ensemble learning algorithm that optimises a customisable loss function by training each successive weak learner to fit the negative gradient of that loss, systematically driving down prediction errors.